EN3160 Assignment 3

October 19, 2026

Index Number: 230724E

Name: Wijesekara WAGS

Github Link: https://github.com/SathsaraGeeth/*vision_a03*.git

1 Modifying the Single-Layer Neural Network

(a) A hidden layer with 100 neurons and sigmoid activation was added between the input and output layers. The forward propagation and manual backpropagation were modified accordingly.

(b) The original mean squared error loss was replaced with cross-entropy loss for multiclass classification.

$$L = -\frac{1}{N}\sum_{i=1}^{N}\log(p_{i,y_i})$$

The gradients were calculated manually using the chain rule.

(c) The network was trained for 10 epochs using a learning rate of 0.01.

- Training accuracy: 37.12%
- Test accuracy: 37.30%
- Initial loss: 2.2078
- Final loss: 1.7925

[6]:
```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# 1. Dataloading
transform = transforms.Compose(
    [transforms.ToTensor(),
```

```
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 50

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
                                         shuffle=False, num_workers=2)

classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')

# 2. Define Network Parameters
Din = 3 * 32 * 32  # Input size (flattened CIFAR-10 image size)
H = 100 # Hidden layer with 100 neurons
K = 10  # Output size (number of classes in CIFAR-10)
std = 0.01 # 1e-5

# Initialize weights and biases
w1 = torch.randn(Din, H) * std  # Input to hidder layer
b1 = torch.zeros(H)
w2 = torch.randn(H, K) * std # Hidden layer to output layer
b2 = torch.zeros(K)

# Hyperparameters
iterations = 10
lr = 0.01 # 2e-6  # Learning rate
lr_decay = 0.9  # Learning rate decay
reg = 0  # Regularization
loss_history = []

# 3. Training Loop
for t in range(iterations):
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # Get inputs and labels
        inputs, labels = data
        Ntr = inputs.shape[0]  # Batch size
        x_train = inputs.view(Ntr, -1)  # Flatten input to (Ntr, Din)
        y_train_onehot = nn.functional.one_hot(labels, K).float()  # Convert␣
 ↪labels to one-hot
```

```
        # Forward pass
        z1 = x_train.mm(w1) + b1
        a1 = torch.sigmoid(z1)
        y_pred = a1.mm(w2) + b2  # Output layer activation

        # Loss calculation (Cross Entropy with regularization)
        loss = nn.functional.cross_entropy(y_pred, labels) + reg * (torch.
 ↪sum(w1 ** 2) + torch.sum(w2 ** 2))
        loss_history.append(loss.item())
        running_loss += loss.item()

        # Backpropagation
        dy_pred = torch.softmax(y_pred, dim=1) - y_train_onehot
        dy_pred /= Ntr

        dw2 = a1.t().mm(dy_pred) + reg * w2
        db2 = dy_pred.sum(dim=0)

        da1 = dy_pred.mm(w2.t())
        dz1 = da1 * a1 * (1 - a1)
        dw1 = x_train.t().mm(dz1) + reg * w1
        db1 = dz1.sum(dim=0)

        # Parameter update
        w1 -= lr * dw1
        b1 -= lr * db1
        w2 -= lr * dw2
        b2 -= lr * db2

    # Print loss for every epoch
    if t % 1 == 0:
        print(f"Epoch {t + 1}/{iterations}, Loss: {running_loss /␣
 ↪len(trainloader)}")

    # Learning rate decay
    lr *= lr_decay

# 4. Plotting the Loss History
plt.plot(loss_history)
plt.title("Loss History")
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.show()

# 5. Calculate Accuracy on Training Set
correct_train = 0
total_train = 0
```

```python
with torch.no_grad():
    for data in trainloader:
        inputs, labels = data
        Ntr = inputs.shape[0]
        x_train = inputs.view(Ntr, -1)
        y_train_onehot = nn.functional.one_hot(labels, K).float()

        # Forward pass
        a1 = torch.sigmoid(x_train.mm(w1) + b1)
        y_train_pred = a1.mm(w2) + b2
        predicted_train = torch.argmax(y_train_pred, dim=1)
        total_train += labels.size(0)
        correct_train += (predicted_train == labels).sum().item()

train_acc = 100 * correct_train / total_train
print(f"Training accuracy: {train_acc:.2f}%")

# 6. Calculate Accuracy on Test Set
correct_test = 0
total_test = 0
with torch.no_grad():
    for data in testloader:
        inputs, labels = data
        Nte = inputs.shape[0]
        x_test = inputs.view(Nte, -1)
        y_test_onehot = nn.functional.one_hot(labels, K).float()

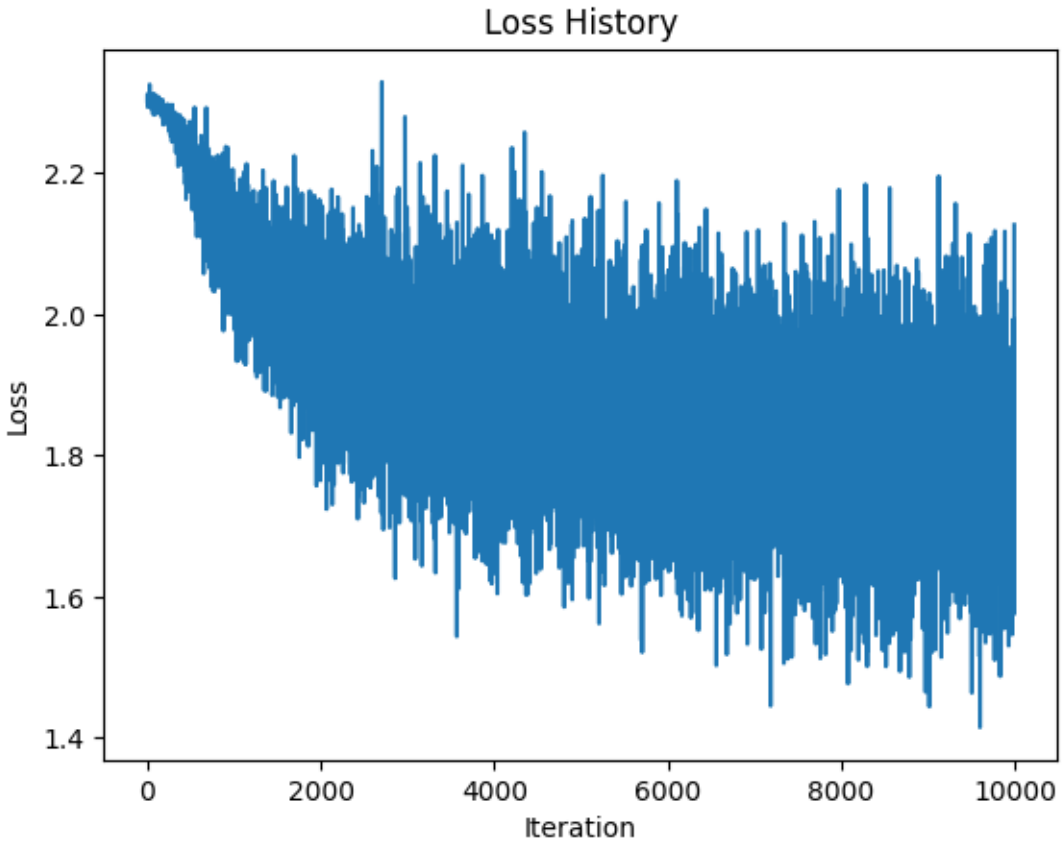
        # Forward pass
        a1 = torch.sigmoid(x_test.mm(w1) + b1)
        y_test_pred = a1.mm(w2) + b2
        predicted_test = torch.argmax(y_test_pred, dim=1)
        total_test += labels.size(0)
        correct_test += (predicted_test == labels).sum().item()

test_acc = 100 * correct_test / total_test
print(f"Test accuracy: {test_acc:.2f}%")
```

```
/home/geeth/miniconda3/envs/yolov13/lib/python3.11/site-
packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype():
align should be passed as Python or NumPy boolean but got `align=0`. Did you
mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
```

```
Epoch 1/10, Loss: 2.207803058385849
Epoch 2/10, Loss: 2.021340085864067
Epoch 3/10, Loss: 1.943696902513504
Epoch 4/10, Loss: 1.9016104465723038
Epoch 5/10, Loss: 1.8719940112829208
```

```
Epoch 6/10, Loss: 1.8488756144046783
Epoch 7/10, Loss: 1.8300811433792115
Epoch 8/10, Loss: 1.8150055284500122
Epoch 9/10, Loss: 1.8027854161262513
Epoch 10/10, Loss: 1.7925096703767776
```




```
Training accuracy: 37.12%
Test accuracy: 37.30%
```

2 Implementing LeNet-5 and a Small 3-Layer CNN Using PyTorch

LeNet-5 and SmallCNN3 were trained on MNIST for 10 epochs under the same conditions. The training and test accuracy and loss were recorded for each epoch, and confusion matrices were generated to evaluate classification performance.

Metric	LeNet-5	SmallCNN3
Training Accuracy	98.53%	98.51%
Test Accuracy	98.25%	97.65%
Training Loss	0.0524	0.0468
Test Loss	0.0615	0.0698

Metric	LeNet-5	SmallCNN3
Training Time	1m 0.9s	2m 25.3s

Both models achieved high classification accuracy with relatively small differences between training and test performance. LeNet-5 performed slightly better in this experiment despite its simpler architecture.

[9]:
```
# LeNet-5 Architecture
# --------------------
#
# Input:    1 x 32 x 32
# Conv1:    6 x 28 x 28   (5x5 convolution)
# SubSamp1: 6 x 14 x 14   (2x2 pooling)
# Conv2:    16 x 10 x 10  (5x5 convolution)
# SubSamp2: 16 x 5 x 5    (2x2 pooling)
# Flatten:  400
# FC1:      120 neurons   (Tanh)
# FC2:      84 neurons    (Tanh)
# Output:   10 neurons    (Gaussian RBF)

class SubsamplingLayer(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.pool = nn.AvgPool2d(kernel_size=2, stride=2)
        self.weight = nn.Parameter(torch.ones(channels))
        self.bias = nn.Parameter(torch.zeros(channels))

    def forward(self, x):
        x = self.pool(x)
        return torch.tanh(x * self.weight.view(1, -1, 1, 1) + self.bias.view(1,␣
 ↪-1, 1, 1))

class GaussianLayer(nn.Module):
    def __init__(self):
        super().__init__()
        self.register_buffer("prototypes", torch.randn(10, 84))

    def forward(self, x):
        return ((x.unsqueeze(1) - self.prototypes) ** 2).sum(dim=2)

class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)
        self.subs1 = SubsamplingLayer(6)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)
```

```
        self.subs2 = SubsamplingLayer(16)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.output = GaussianLayer()

    def forward(self, x):
        x = torch.tanh(self.conv1(x))
        x = self.subs1(x)
        x = torch.tanh(self.conv2(x))
        x = self.subs2(x)
        x = torch.flatten(x, 1)
        x = torch.tanh(self.fc1(x))
        x = torch.tanh(self.fc2(x))
        x = self.output(x)
        return -x
```

[19]:
```
# SmallCNN3 Architecture
# ----------------------
#
# Input:    1 x 32 x 32
# Conv1:    32 x 32 x 32   (3x3 convolution, BatchNorm, ReLU)
# MaxPool1: 32 x 16 x 16   (2x2 pooling)
# Conv2:    64 x 16 x 16   (3x3 convolution, BatchNorm, ReLU)
# MaxPool2: 64 x 8 x 8     (2x2 pooling)
# Conv3:    128 x 8 x 8    (3x3 convolution, BatchNorm, ReLU)
# AvgPool:  128 x 1 x 1    (Global Average Pooling)
# Flatten:  128
# Dropout:  0.5
# FC:       10 neurons

class SmallCNN3(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.dropout = nn.Dropout(0.5)
        self.fc = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.bn1(self.conv1(x))))
        x = self.pool(torch.relu(self.bn2(self.conv2(x))))
```

```
        x = torch.relu(self.bn3(self.conv3(x)))
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.dropout(x)
        x = self.fc(x)
        return x
```

[17]:
```
# Load the dataset
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

transform = transforms.Compose([
    transforms.Pad(2),
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

batch_size = 50

trainset = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)

testset = torchvision.datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)

trainloader = torch.utils.data.DataLoader(
    trainset, batch_size=batch_size, shuffle=True, num_workers=2
)

testloader = torch.utils.data.DataLoader(
    testset, batch_size=batch_size, shuffle=False, num_workers=2
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

[ ]:
```
# Train and Evaluation
def evaluate(model, loader, criterion):
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0
```

```
    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * labels.size(0)
            predicted = torch.argmax(outputs, dim=1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    return running_loss / total, 100 * correct / total


def train_model(model, epochs=10):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    history = {
        "train_loss": [],
        "test_loss": [],
        "train_acc": [],
        "test_acc": []
    }

    for epoch in range(epochs):
        model.train()

        for inputs, labels in trainloader:
            inputs = inputs.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        train_loss, train_acc = evaluate(model, trainloader, criterion)
        test_loss, test_acc = evaluate(model, testloader, criterion)

        history["train_loss"].append(train_loss)
        history["test_loss"].append(test_loss)
        history["train_acc"].append(train_acc)
        history["test_acc"].append(test_acc)
```

```
        print(
            f"Epoch {epoch + 1}/{epochs}, "
            f"Train Loss: {train_loss:.4f}, "
            f"Test Loss: {test_loss:.4f}, "
            f"Train Acc: {train_acc:.2f}%, "
            f"Test Acc: {test_acc:.2f}%"
        )

    return model, history
```

[13]:
```
# Plot
def plot_history(history, title):
    epochs = range(1, len(history["train_loss"]) + 1)

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(epochs, history["train_loss"], label="Train")
    plt.plot(epochs, history["test_loss"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"{title} Loss")
    plt.legend()

    plt.subplot(1, 2, 2)
    plt.plot(epochs, history["train_acc"], label="Train")
    plt.plot(epochs, history["test_acc"], label="Test")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy (%)")
    plt.title(f"{title} Accuracy")
    plt.legend()

    plt.tight_layout()
    plt.show()
```

[ ]:
```
# Confusion Matrix
def plot_confusion_matrix(model, loader, title):
    model.eval()

    cm = torch.zeros(10, 10, dtype=torch.int64)

    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(device)
            outputs = model(inputs)
            predicted = torch.argmax(outputs, dim=1).cpu()
```

```
            for actual, pred in zip(labels, predicted):
                cm[actual, pred] += 1

    plt.figure(figsize=(8, 6))
    plt.imshow(cm, cmap="Blues")
    plt.colorbar()

    plt.xticks(range(10))
    plt.yticks(range(10))
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.title(f"{title} Confusion Matrix")

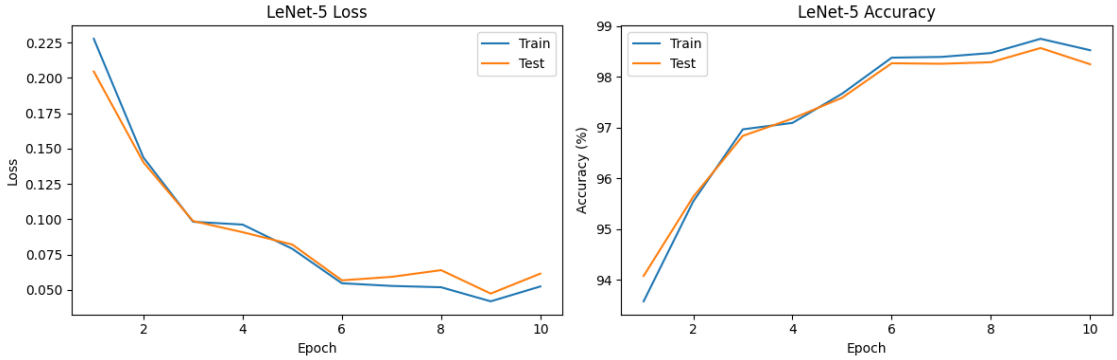
    for i in range(10):
        for j in range(10):
            plt.text(j, i, cm[i, j].item(), ha="center", va="center",␣
↪fontsize=8)

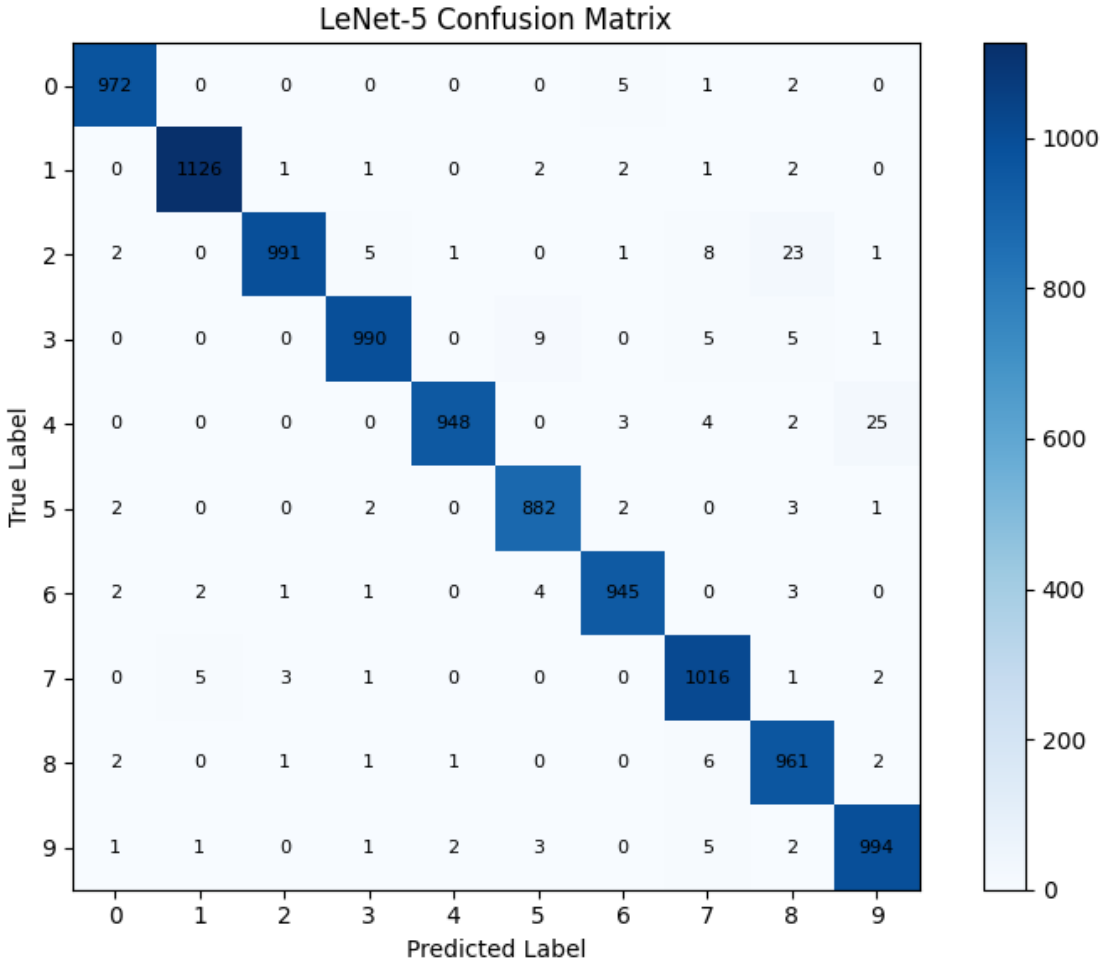
    plt.tight_layout()
    plt.show()
```

[16]:
```
# Train LeNet-5
lenet = LeNet5()
lenet, lenet_history = train_model(lenet, epochs=10)
plot_history(lenet_history, "LeNet-5")
plot_confusion_matrix(lenet, testloader, "LeNet-5")
print(f"LeNet-5 Training Accuracy: {lenet_history['train_acc'][-1]:.2f}%")
print(f"LeNet-5 Test Accuracy: {lenet_history['test_acc'][-1]:.2f}%")
```

```
Epoch 1/10, Train Loss: 0.2276, Test Loss: 0.2045, Train Acc: 93.58%, Test Acc:
94.08%
Epoch 2/10, Train Loss: 0.1436, Test Loss: 0.1402, Train Acc: 95.55%, Test Acc:
95.64%
Epoch 3/10, Train Loss: 0.0982, Test Loss: 0.0987, Train Acc: 96.97%, Test Acc:
96.84%
Epoch 4/10, Train Loss: 0.0962, Test Loss: 0.0908, Train Acc: 97.09%, Test Acc:
97.18%
Epoch 5/10, Train Loss: 0.0790, Test Loss: 0.0821, Train Acc: 97.67%, Test Acc:
97.59%
Epoch 6/10, Train Loss: 0.0546, Test Loss: 0.0567, Train Acc: 98.38%, Test Acc:
98.27%
Epoch 7/10, Train Loss: 0.0527, Test Loss: 0.0592, Train Acc: 98.39%, Test Acc:
98.26%
Epoch 8/10, Train Loss: 0.0518, Test Loss: 0.0639, Train Acc: 98.47%, Test Acc:
98.29%
Epoch 9/10, Train Loss: 0.0419, Test Loss: 0.0473, Train Acc: 98.75%, Test Acc:
98.57%
Epoch 10/10, Train Loss: 0.0524, Test Loss: 0.0615, Train Acc: 98.53%, Test Acc:
```

98.25%

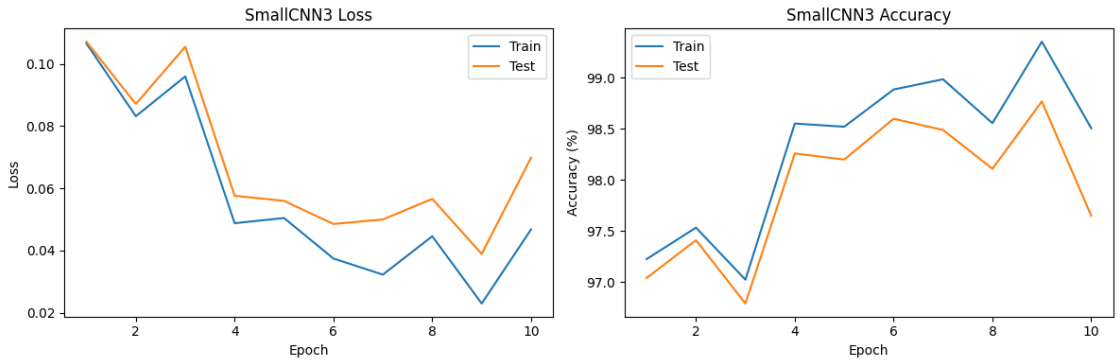


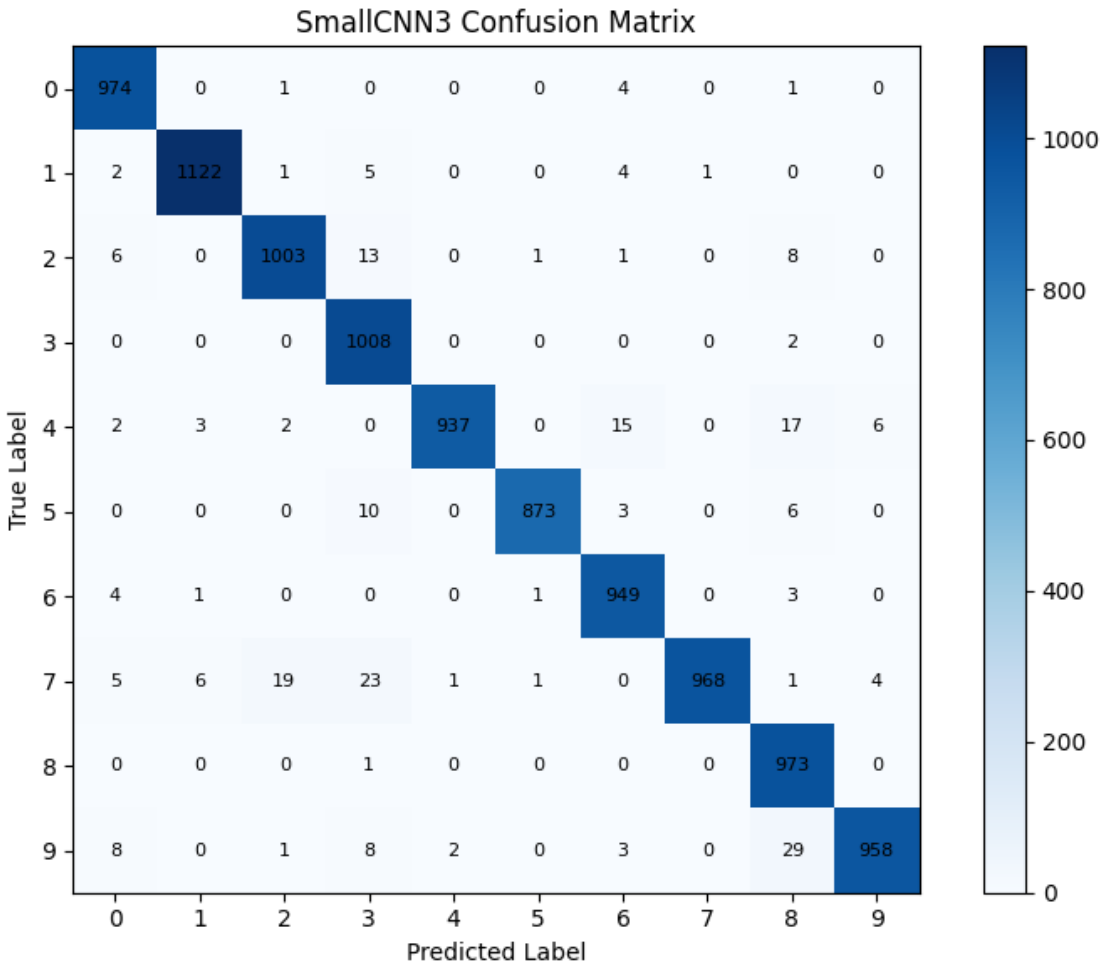


LeNet-5 Training Accuracy: 98.53%
LeNet-5 Test Accuracy: 98.25%

[20]:
```
# Train SmallCNN3
smallcnn = SmallCNN3()
smallcnn, smallcnn_history = train_model(smallcnn, epochs=10)
plot_history(smallcnn_history, "SmallCNN3")
plot_confusion_matrix(smallcnn, testloader, "SmallCNN3")
print(f"SmallCNN3 Training Accuracy: {smallcnn_history['train_acc'][-1]:.2f}%")
print(f"SmallCNN3 Test Accuracy: {smallcnn_history['test_acc'][-1]:.2f}%")
```

```
Epoch 1/10, Train Loss: 0.1065, Test Loss: 0.1071, Train Acc: 97.22%, Test Acc:
97.04%
Epoch 2/10, Train Loss: 0.0831, Test Loss: 0.0871, Train Acc: 97.53%, Test Acc:
97.41%
Epoch 3/10, Train Loss: 0.0959, Test Loss: 0.1055, Train Acc: 97.02%, Test Acc:
96.79%
Epoch 4/10, Train Loss: 0.0488, Test Loss: 0.0576, Train Acc: 98.55%, Test Acc:
98.26%
Epoch 5/10, Train Loss: 0.0504, Test Loss: 0.0559, Train Acc: 98.52%, Test Acc:
98.20%
Epoch 6/10, Train Loss: 0.0374, Test Loss: 0.0485, Train Acc: 98.89%, Test Acc:
98.60%
Epoch 7/10, Train Loss: 0.0322, Test Loss: 0.0500, Train Acc: 98.99%, Test Acc:
98.49%
Epoch 8/10, Train Loss: 0.0446, Test Loss: 0.0566, Train Acc: 98.56%, Test Acc:
98.11%
Epoch 9/10, Train Loss: 0.0229, Test Loss: 0.0389, Train Acc: 99.36%, Test Acc:
98.77%
Epoch 10/10, Train Loss: 0.0468, Test Loss: 0.0698, Train Acc: 98.51%, Test Acc:
97.65%
```






```
SmallCNN3 Training Accuracy: 98.51%
SmallCNN3 Test Accuracy: 97.65%
```

3 Transfer Learning Using Pre-trained ResNet-18

A ResNet-18 model pretrained on ImageNet-1K was used to classify the Hymenoptera dataset using two approaches.

(a) In fine-tuning, all pretrained layers were updated during training.

(b) In feature extraction, the pretrained layers were frozen, and only the final classification layer was trained.

Both models were trained for 25 epochs.

Metric	Fine-Tuning	Feature Extraction
Best Validation Accuracy	96.08%	94.77%
Final Training Accuracy	86.07%	81.56%

Metric	Fine-Tuning	Feature Extraction
Final Validation Accuracy	94.12%	92.81%

Both approaches achieved good classification accuracy. Fine-tuning performed slightly better, achieving a best validation accuracy of 96.08%, compared to 94.77% for feature extraction. However, feature extraction required less computation since only the final layer was trained.

[31]:
```
# Load ResNet18 pretrained on ImageNet-1K
import torch
import torch.nn as nn
import torch.optim as optim
from torch.optim import lr_scheduler
from torchvision import datasets, transforms
from torchvision.models import resnet18, ResNet18_Weights
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import os
import copy
import time

weights = ResNet18_Weights.IMAGENET1K_V1
model = resnet18(weights=weights)
preprocess = weights.transforms()
model.eval();
```

[ ]:
```
# Load Hymenoptrea Dataset
import urllib.request
import tarfile
import zipfile

url = "https://download.pytorch.org/tutorial/hymenoptera_data.zip"
urllib.request.urlretrieve(url, "hymenoptera_data.zip")

with zipfile.ZipFile("hymenoptera_data.zip", "r") as zip_ref:
    zip_ref.extractall(".")

data_transforms = {
    'train': transforms.Compose([
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ]),
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
```

```
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])
}

data_dir = './hymenoptera_data'

image_datasets = {
    x: datasets.ImageFolder(os.path.join(data_dir, x), data_transforms[x])
    for x in ['train', 'val']
}

dataloaders = {
    x: DataLoader(image_datasets[x], batch_size=4, shuffle=True, num_workers=2)
    for x in ['train', 'val']
}

dataset_sizes = {x: len(image_datasets[x]) for x in ['train', 'val']}
class_names = image_datasets['train'].classes
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(class_names)
print(device)
```

```
['ants', 'bees']
cpu
```

[35]:
```
# Training
def train_model(model, criterion, optimizer, scheduler, num_epochs=25):
    best_model_wts = copy.deepcopy(model.state_dict())
    best_acc = 0.0

    for epoch in range(num_epochs):
        print(f'Epoch {epoch + 1}/{num_epochs}')

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

            running_loss = 0.0
            running_corrects = 0

            for inputs, labels in dataloaders[phase]:
                inputs = inputs.to(device)
                labels = labels.to(device)
```

```
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    loss = criterion(outputs, labels)
                    preds = torch.argmax(outputs, dim=1)

                    if phase == 'train':
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += (preds == labels).sum().item()

            if phase == 'train':
                scheduler.step()

            epoch_loss = running_loss / dataset_sizes[phase]
            epoch_acc = running_corrects / dataset_sizes[phase]

            print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

            if phase == 'val' and epoch_acc > best_acc:
                best_acc = epoch_acc
                best_model_wts = copy.deepcopy(model.state_dict())

    model.load_state_dict(best_model_wts)
    return model
```

[36]:
```
# (a) Fine-Tuning ResNet-18
model_ft = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
num_ftrs = model_ft.fc.in_features
model_ft.fc = nn.Linear(num_ftrs, 2)
model_ft = model_ft.to(device)
criterion = nn.CrossEntropyLoss()
optimizer_ft = optim.SGD(model_ft.parameters(), lr=0.001, momentum=0.9)
scheduler_ft = lr_scheduler.StepLR(optimizer_ft, step_size=7, gamma=0.1)
model_ft = train_model(model_ft, criterion, optimizer_ft, scheduler_ft,␣
 ↪num_epochs=25)
```

```
Epoch 1/25

train Loss: 0.5380 Acc: 0.7213
val Loss: 0.2668 Acc: 0.8758
Epoch 2/25
train Loss: 0.7234 Acc: 0.7500
val Loss: 0.8969 Acc: 0.7582
```

```
Epoch 3/25
train Loss: 0.7044 Acc: 0.7869
val Loss: 0.3336 Acc: 0.9020
Epoch 4/25
train Loss: 0.6259 Acc: 0.7459
val Loss: 0.3995 Acc: 0.8431
Epoch 5/25
train Loss: 0.7024 Acc: 0.7541
val Loss: 0.3430 Acc: 0.8954
Epoch 6/25
train Loss: 0.6258 Acc: 0.8074
val Loss: 0.6067 Acc: 0.8105
Epoch 7/25
train Loss: 0.3602 Acc: 0.8566
val Loss: 0.3984 Acc: 0.9085
Epoch 8/25
train Loss: 0.4227 Acc: 0.8402
val Loss: 0.2786 Acc: 0.9412
Epoch 9/25
train Loss: 0.2415 Acc: 0.9016
val Loss: 0.2703 Acc: 0.9346
Epoch 10/25
train Loss: 0.3468 Acc: 0.8689
val Loss: 0.2563 Acc: 0.9412
Epoch 11/25
train Loss: 0.3371 Acc: 0.8525
val Loss: 0.2502 Acc: 0.9412
Epoch 12/25
train Loss: 0.2537 Acc: 0.8730
val Loss: 0.2383 Acc: 0.9542
Epoch 13/25
train Loss: 0.3459 Acc: 0.8607
val Loss: 0.2314 Acc: 0.9477
Epoch 14/25
train Loss: 0.3760 Acc: 0.8402
val Loss: 0.2170 Acc: 0.9542
Epoch 15/25
train Loss: 0.3867 Acc: 0.8115
val Loss: 0.2170 Acc: 0.9412
Epoch 16/25
train Loss: 0.2432 Acc: 0.9057
val Loss: 0.2201 Acc: 0.9346
Epoch 17/25
train Loss: 0.2386 Acc: 0.8934
val Loss: 0.2125 Acc: 0.9542
Epoch 18/25
train Loss: 0.3323 Acc: 0.8402
val Loss: 0.2147 Acc: 0.9608
```

```
Epoch 19/25
train Loss: 0.3193 Acc: 0.8443
val Loss: 0.2306 Acc: 0.9542
Epoch 20/25
train Loss: 0.3258 Acc: 0.8648
val Loss: 0.2250 Acc: 0.9477
Epoch 21/25
train Loss: 0.2752 Acc: 0.8811
val Loss: 0.2238 Acc: 0.9477
Epoch 22/25
train Loss: 0.2061 Acc: 0.9016
val Loss: 0.2095 Acc: 0.9477
Epoch 23/25
train Loss: 0.2353 Acc: 0.9139
val Loss: 0.2336 Acc: 0.9412
Epoch 24/25
train Loss: 0.2910 Acc: 0.8689
val Loss: 0.2285 Acc: 0.9477
Epoch 25/25
train Loss: 0.3138 Acc: 0.8607
val Loss: 0.2285 Acc: 0.9412
```

[37]:
```
# ResNet-18 as a Feature Extractor
model_conv = resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)

for param in model_conv.parameters():
    param.requires_grad = False

num_ftrs = model_conv.fc.in_features
model_conv.fc = nn.Linear(num_ftrs, 2)
model_conv = model_conv.to(device)
criterion = nn.CrossEntropyLoss()
optimizer_conv = optim.SGD(model_conv.fc.parameters(), lr=0.001, momentum=0.9)
scheduler_conv = lr_scheduler.StepLR(optimizer_conv, step_size=7, gamma=0.1)
model_conv = train_model(model_conv, criterion, optimizer_conv, scheduler_conv,␣
 ↪num_epochs=25)
```

```
Epoch 1/25
train Loss: 0.5192 Acc: 0.7131
val Loss: 0.4695 Acc: 0.7712
Epoch 2/25
train Loss: 0.5325 Acc: 0.7418
val Loss: 0.1756 Acc: 0.9477
Epoch 3/25
train Loss: 0.4423 Acc: 0.8033
val Loss: 0.2207 Acc: 0.9216
Epoch 4/25
train Loss: 0.4714 Acc: 0.7623
```

```
val Loss: 0.3220 Acc: 0.8627
Epoch 5/25
train Loss: 0.4585 Acc: 0.8238
val Loss: 0.2739 Acc: 0.8889
Epoch 6/25
train Loss: 0.6100 Acc: 0.7541
val Loss: 0.2032 Acc: 0.9281
Epoch 7/25
train Loss: 0.4714 Acc: 0.8033
val Loss: 0.1963 Acc: 0.9412
Epoch 8/25
train Loss: 0.3189 Acc: 0.8730
val Loss: 0.1722 Acc: 0.9412
Epoch 9/25
train Loss: 0.4339 Acc: 0.7828
val Loss: 0.1842 Acc: 0.9477
Epoch 10/25
train Loss: 0.3470 Acc: 0.8525
val Loss: 0.1840 Acc: 0.9477
Epoch 11/25
train Loss: 0.2970 Acc: 0.8689
val Loss: 0.1835 Acc: 0.9412
Epoch 12/25
train Loss: 0.4307 Acc: 0.7992
val Loss: 0.1766 Acc: 0.9412
Epoch 13/25
train Loss: 0.2884 Acc: 0.8893
val Loss: 0.1777 Acc: 0.9412
Epoch 14/25
train Loss: 0.3538 Acc: 0.8320
val Loss: 0.1955 Acc: 0.9412
Epoch 15/25
train Loss: 0.3651 Acc: 0.8115
val Loss: 0.2524 Acc: 0.9150
Epoch 16/25
train Loss: 0.2637 Acc: 0.8893
val Loss: 0.1794 Acc: 0.9346
Epoch 17/25
train Loss: 0.2609 Acc: 0.8893
val Loss: 0.1860 Acc: 0.9346
Epoch 18/25
train Loss: 0.3386 Acc: 0.8566
val Loss: 0.1865 Acc: 0.9412
Epoch 19/25
train Loss: 0.2820 Acc: 0.8730
val Loss: 0.1886 Acc: 0.9412
Epoch 20/25
train Loss: 0.3877 Acc: 0.8197
```

```
val Loss: 0.1928 Acc: 0.9346
Epoch 21/25
train Loss: 0.3311 Acc: 0.8566
val Loss: 0.1820 Acc: 0.9412
Epoch 22/25
train Loss: 0.3617 Acc: 0.8320
val Loss: 0.2084 Acc: 0.9085
Epoch 23/25
train Loss: 0.3181 Acc: 0.8648
val Loss: 0.2060 Acc: 0.9281
Epoch 24/25
train Loss: 0.4314 Acc: 0.8115
val Loss: 0.1670 Acc: 0.9477
Epoch 25/25
train Loss: 0.3827 Acc: 0.8156
val Loss: 0.1899 Acc: 0.9281
```